Ex.No : 8 Perform various testing such as functional, usability, interface using testing tools such as selenium

Date:

Aim:

To Perform various testing such as functional, usability, interface using testing tools such as selenium

Procedure:

Technologies Used:

- **Express.js**: For handling HTTP requests and routing.
- **MongoDB + MongoClient**: For interacting with a cloud MongoDB database (via Atlas).
- **CORS & body-parser**: Middleware to handle cross-origin requests and JSON body parsing.
- **Selenium:** Open-source software framework for automating web applications

Testing:

- **Functional Interface Testing**: Verifies that all UI elements (buttons, forms, links) work as intended according to requirements.
- **Usability Testing:** Evaluates how easy and intuitive the interface is for users to accomplish tasks.
- **Interface Testing**: Verifies that different software modules or systems interact correctly through their interface, ensuring data is passed accurately and functions are executed as expected.

Code:

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(express.json());
app.use(cors());

// Connect to MongoDB Atlas
mongoose.connect(
      "mongodb+srv://tceaswin:Aswin123@cluster0.fltbugm.mongodb.net/boookngo?retryWrites=true&w=majority",
   { useNewUrlParser: true, useUnifiedTopology: true }
)
   .then(() => console.log("Connected to MongoDB Atlas"))
   .catch((err) => console.error("MongoDB connection error:", err));
```

```
const bookingSchema = new mongoose.Schema({
    name: String,
    email: String,
    phone: String,
    city: String,
    date: String,
    hotel: String,
    createdAt: { type: Date, default: Date.now }
});

const Book = mongoose.model("Book", bookingSchema);

// Transport Booking Schema
const transportSchema = new mongoose.Schema({
    name: String,
    type: String,
    from: String,
    to: String,
    time: String,
    createdAt: { type: Date, default: Date.now }
});

const Transport = mongoose.model("Transport", transportSchema);

// CREATE
app.post("/api/bookings", async (req, res) => {
    try {
        const newBooking = new Book(req.body);
        await newBooking.save();
        res.status(201).json({ message: "Booking Saved", booking: newBooking });
    } catch (err) {
        res.status(500).json({ message: "Error", err });
    }
});

// READ ALL
app.get("/api/bookings", async (req, res) => {
    try {
        const bookings = await Book.find();
        res.json(bookings);
    } catch (err) {
        res.status(500).json({ message: "Error", err });
    }
});
```

```
// READ ONE
app.get("/api/bookings/:id", async (req, res) => {
   try {
      const booking = await Book.findById(req.params.id);
      if (!booking) return res.status(404).json({ message: "Not found" });
      res.json(booking);
   } catch (err) {
      res.status(500).json({ message: "Error", err });
   }
});

// UPDATE
app.put("/api/bookings/:id", async (req, res) => {
   try {
      const updated = await Book.findByIdAndUpdate(req.params.id, req.body, { new: true });
      if (!updated) return res.status(404).json({ message: "Not found" });
      res.json({ message: "Booking Updated", booking: updated });
   } catch (err) {
      res.status(500).json({ message: "Error", err });
   }
});

// DELETE
app.delete("/api/bookings/:id", async (req, res) => {
   try {
      const deleted = await Book.findByIdAndDelete(req.params.id);
      if (!deleted) return res.status(404).json({ message: "Not found" });
      res.json({ message: "Booking Deleted" });
   } catch (err) {
      res.status(500).json({ message: "Error", err });
   }
});



// TRANSPORT CRUD ENDPOINTS
// CREATE
app.post("/api/transports", async (req, res) => {
   try {
      const newTransport = new Transport(req.body);
      await newTransport.save();
      res.status(201).json({ message: "Transport Booking Saved", transport: newTransport });
   } catch (err) {
      res.status(500).json({ message: "Error", err });
   }
```

```
});

// READ ALL
app.get("/api/transports", async (req, res) => {
  try {
    const transports = await Transport.find();
    res.json(transports);
  } catch (err) {
    res.status(500).json({ message: "Error", err });
  }
});

// READ ONE
app.get("/api/transports/:id", async (req, res) => {
  try {
    const transport = await Transport.findById(req.params.id);
    if (!transport) return res.status(404).json({ message: "Not found" });
    res.json(transport);
  } catch (err) {
    res.status(500).json({ message: "Error", err });
  }
});

// UPDATE
app.put("/api/transports/:id", async (req, res) => {
  try {
    const updated = await Transport.findByIdAndUpdate(req.params.id, req.body, { new: true });
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json({ message: "Transport Booking Updated", transport: updated });
  } catch (err) {
    res.status(500).json({ message: "Error", err });
  }
});

// DELETE
app.delete("/api/transports/:id", async (req, res) => {
  try {
    const deleted = await Transport.findByIdAndDelete(req.params.id);
    if (!deleted) return res.status(404).json({ message: "Not found" });
    res.json({ message: "Transport Booking Deleted" });
  } catch (err) {
    res.status(500).json({ message: "Error", err });
  }
});
```

```
// Root route for API status
app.get('/', (req, res) => {
   res.send('BookNGo API is running. Available endpoints: \n' +
      'POST /api/bookings, GET /api/bookings, GET /api/bookings/:id, PUT /api/bookings/:id, DELETE
/api/bookings/:id\n' +
      'POST /api/transports, GET /api/transports, GET /api/transports/:id, PUT /api/transports/:id, DELETE
/api/transports/:id');
});

app.listen(5000, () => {
   console.log("Server running on port 5000");
});
```

selenium.test.js:

```
// Selenium test for BookNGo hotel booking page
const { Builder, By, until } = require('selenium-webdriver');
const chrome = require('selenium-webdriver/chrome');

(async function bookingTest() {
  // Set up Chrome driver
  let driver = await new Builder().forBrowser('chrome').setChromeOptions(new chrome.Options()).build();
  try {
    // Open the booking page (update path if needed)
    await driver.get('file://' + __dirname + '/../html/booking.html');

    // Wait for the form to be present
    await driver.wait(until.elementLocated(By.css('form')), 5000);

    // Fill the form fields
    await driver.findElement(By.css('input[ng-model="vm.form.name"]')).sendKeys('Test User');
    await driver.findElement(By.css('input[ng-model="vm.form.email"]')).sendKeys('test@example.com');
    await driver.findElement(By.css('input[ng-model="vm.form.phone"]')).sendKeys('1234567890');
    await driver.findElement(By.css('input[ng-model="vm.form.city"]')).sendKeys('Test City');
    await driver.findElement(By.css('input[ng-model="vm.form.date"]')).sendKeys('2025-09-30');
    await driver.findElement(By.css('input[ng-model="vm.form.hotel"]')).sendKeys('Test Hotel');

    // Submit the form
    await driver.findElement(By.css('button[type="submit"]')).click();

    // Wait for success message
    await driver.wait(until.elementLocated(By.css('.alert-success')), 5000);
    let successMsg = await driver.findElement(By.css('.alert-success')).getText();
    console.log('Success message:', successMsg);
  } catch (err) {
```

```
    console.error('Test failed:', err);
  } finally {
    await driver.quit();
  }
})();

// Selenium test for BookNGo transport booking page
(async function transportTest() {
  let driver = await new Builder().forBrowser('chrome').setChromeOptions(new chrome.Options()).build();
  try {
    console.log('Opening transport booking page...');
    await driver.get('file://' + __dirname + '/../html/transport.html');

    console.log('Waiting for form to be present...');
    await driver.wait(until.elementLocated(By.css('form')), 5000);

    // Test 1: Valid input
    console.log('Test 1: Filling valid transport booking form...');
    await driver.findElement(By.css('input[ng-model="vm.form.name"]')).sendKeys('Test Transport');
    await driver.findElement(By.css('select[ng-model="vm.form.type"]')).sendKeys('Bus');
    await driver.findElement(By.css('input[ng-model="vm.form.from"]')).sendKeys('City A');
    await driver.findElement(By.css('input[ng-model="vm.form.to"]')).sendKeys('City B');
    await driver.findElement(By.css('input[ng-model="vm.form.time"]')).sendKeys('10:00 AM');
    console.log('Submitting valid form...');
    await driver.findElement(By.css('button[type="submit"]')).click();
    console.log('Waiting for success message...');
    await driver.wait(until.elementLocated(By.css('.alert-success')), 5000);
    let successMsg = await driver.findElement(By.css('.alert-success')).getText();
    console.log('Transport Success message:', successMsg);

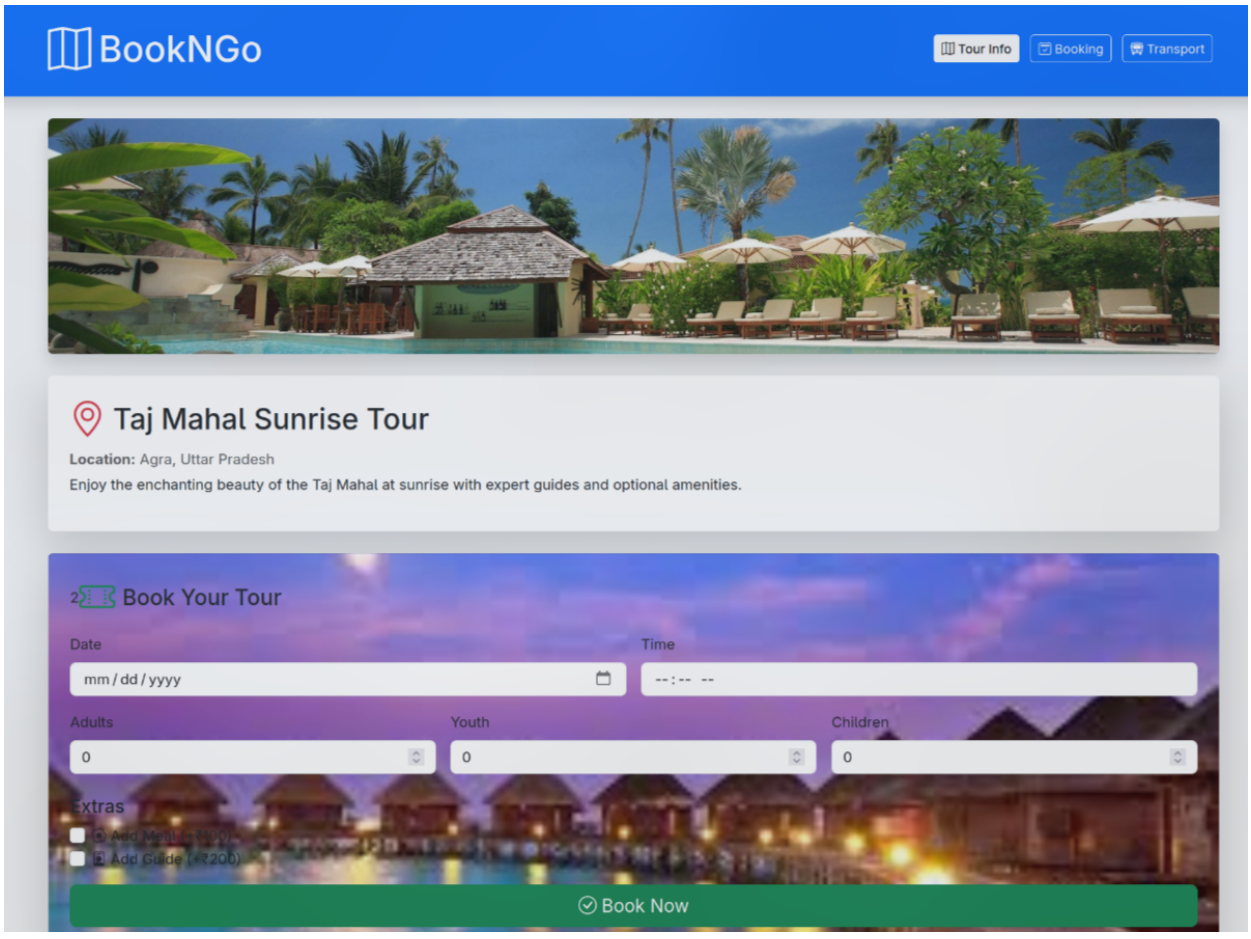
    // Test 2: Invalid input (missing fields)
    console.log('Test 2: Submitting form with missing fields...');
    await driver.navigate().refresh();
    await driver.wait(until.elementLocated(By.css('form')), 5000);
    await driver.findElement(By.css('button[type="submit"]')).click();
    await driver.wait(until.elementLocated(By.css('.alert-danger')), 5000);
    let errorMsg1 = await driver.findElement(By.css('.alert-danger')).getText();
    console.log('Error message (missing fields):', errorMsg1);

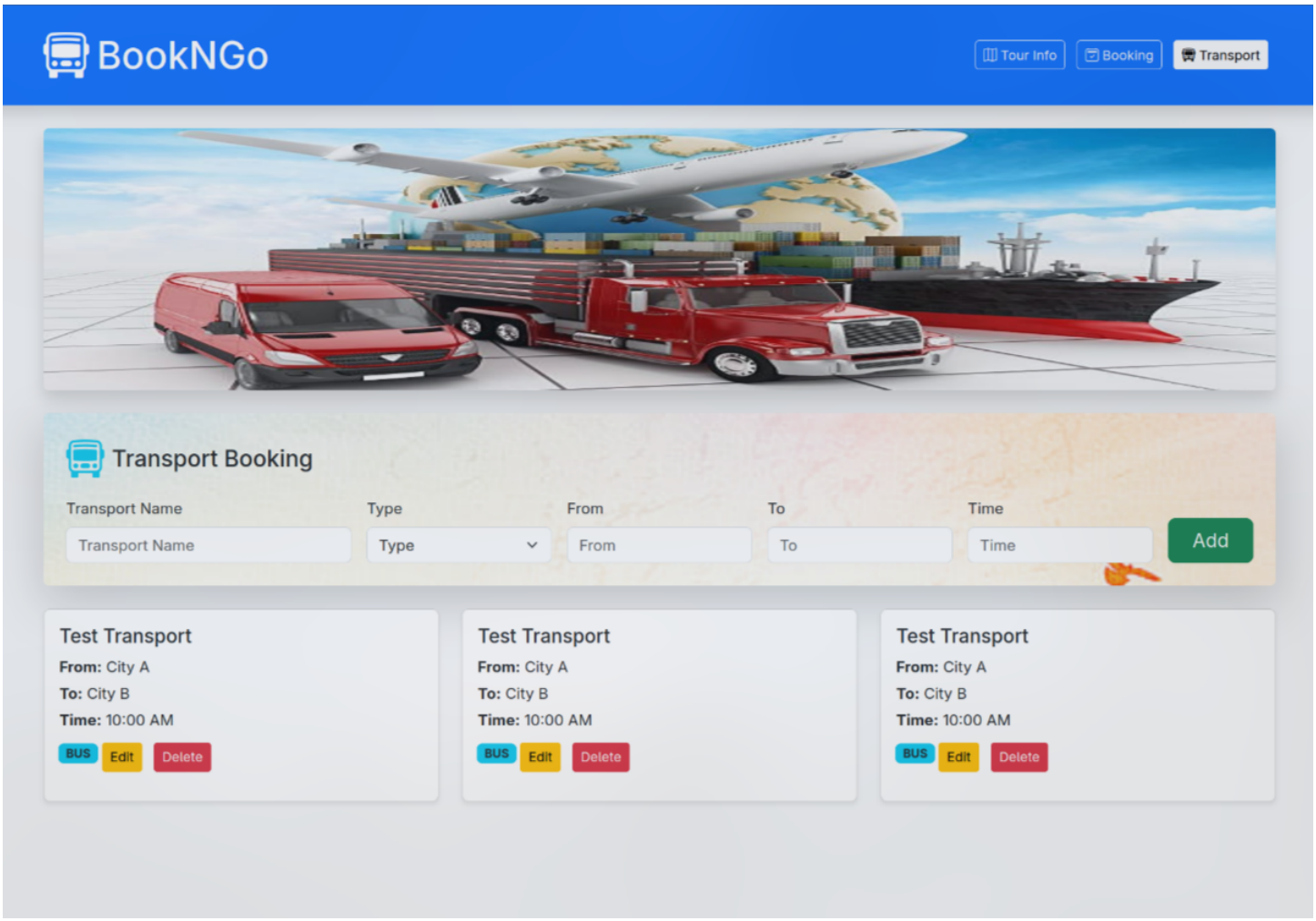
    // Test 3: Invalid input (bad time format)
    console.log('Test 3: Submitting form with invalid time format...');
    await driver.findElement(By.css('input[ng-model="vm.form.name"]')).sendKeys('Test Transport');
    await driver.findElement(By.css('select[ng-model="vm.form.type"]')).sendKeys('Bus');
    await driver.findElement(By.css('input[ng-model="vm.form.from"]')).sendKeys('City A');
    await driver.findElement(By.css('input[ng-model="vm.form.to"]')).sendKeys('City B');
```

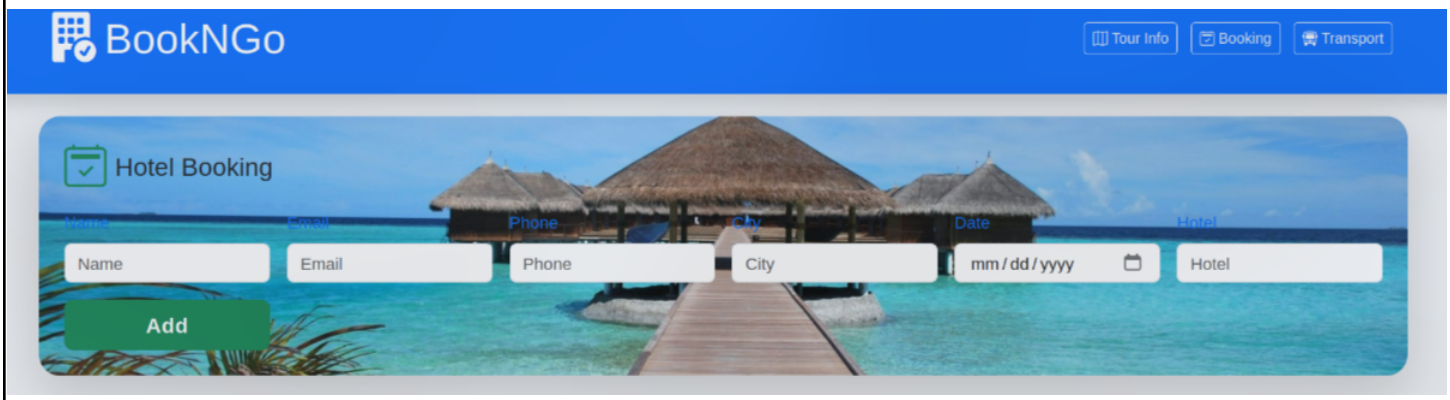
```
    await driver.findElement(By.css('input[ng-model="vm.form.time"]')).sendKeys('invalidtime');
    await driver.findElement(By.css('button[type="submit"]')).click();
    await driver.wait(until.elementLocated(By.css('.alert-danger')), 5000);
    let errorMsg2 = await driver.findElement(By.css('.alert-danger')).getText();
    console.log('Error message (invalid time):', errorMsg2);

  } catch (err) {
    console.error('Transport Test failed:', err);
  } finally {
    console.log('Closing browser...');
    await driver.quit();
  }
})();
```

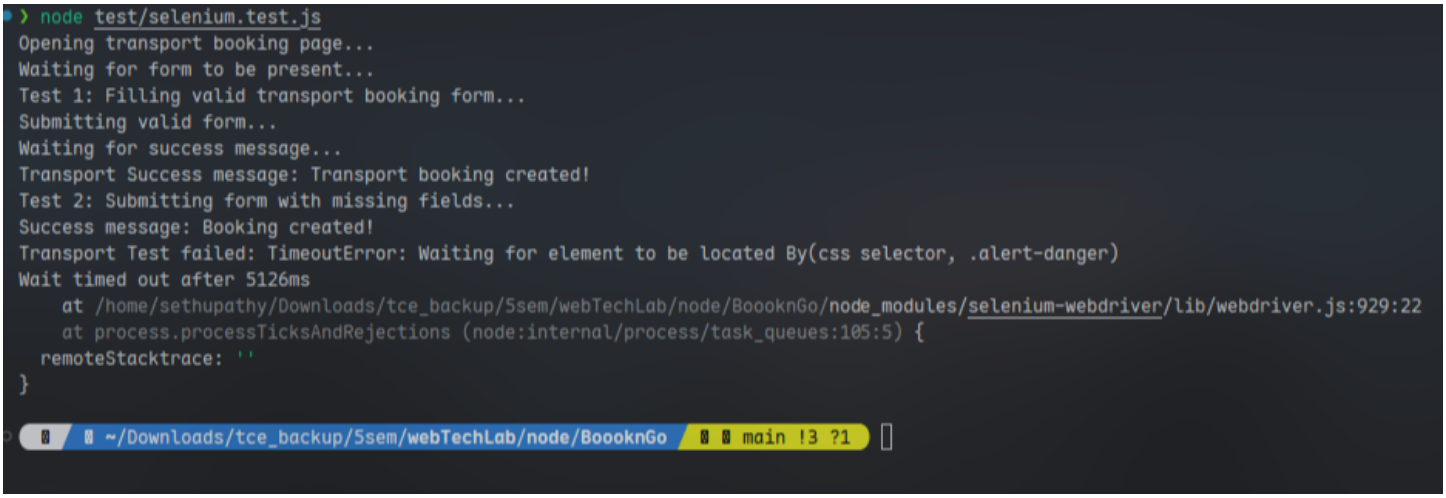
Output:




BookNGo
Tour Info
Booking
Transport
Transport Booking
Transport Name
Type
From
To
Time
Transport Name
Type
From
To
Time
Add
Test Transport
From: City A
To: City B
Time: 10:00 AM
BUS
Edit
Delete
Test Transport
From: City A
To: City B
Time: 10:00 AM
BUS
Edit
Delete
Test Transport
From: City A
To: City B
Time: 10:00 AM
BUS
Edit
Delete


BookNGo
Tour Info
Booking
Transport
Hotel Booking
Name
Email
Phone
City
Date
Hotel
Name
Email
Phone
City
mm / dd / yyyy
Hotel
Add

Testing results:



Result:

Thus, the functional, usability and interface testing using testing tool selenium was performed successfully.